SDG BLOCKCHAIN ACCELERATOR

Technical Architecture Document

In collaboration with

UNDP Bosnia and Herzegovina / University of Sarajevo) / Sala

1. Project Information

Project Name: Digital Identity & Credential Verification
Challenge & UNDP Office: UNDP Bosnia and Herzegovina
Challenge Owner: University of Sarajevo
Document Version: v1.1 (Enhanced)

2. Overview

This project implements a decentralized system for issuing and verifying academic credentials using the Cardano blockchain. The goal is to eliminate reliance on centralized verification processes, which are often slow, opaque, and prone to fraud.

The system introduces a verifiable credential model where institutions issue digitally signed certificates whose cryptographic proofs are anchored on-chain. Sensitive data remains off-chain, ensuring compliance with privacy requirements while maintaining verifiability.

At a high level, the platform enables:

- Educational institutions to issue tamper-proof credentials
- Students to own and share credentials via QR codes
- Third parties to verify authenticity independently, without contacting the issuer

The architecture follows a **minimal on-chain footprint strategy**:

- Only hashes and verification references are stored on-chain
- Full credential data is stored off-chain (IPFS / database)
- Verification is performed through hash matching and signature validation

This approach ensures scalability while preserving trust guarantees.

The solution contributes to:

- **SDG 4 (Quality Education):** Trusted and portable credentials
- **SDG 16 (Strong Institutions):** Transparent verification processes
- **SDG 9 (Innovation & Infrastructure):** Digital trust infrastructure
- **SDG 10 (Reduced Inequalities):** Cross-border credential recognition

3. System Architecture

3.1 High-Level Architecture

```
┌──────────────────────────────────┐
│        User Interface            │
│  (Students / Universities)       │
└──────────────────────────────────┘
                │
      Wallet Connection (CIP-30)
                │
┌───────────────▼──────────────────┐
│   Transaction Builder            │
│   (Lucid / Off-chain SDK)        │
└──────────────────────────────────┘
                │
┌───────────────▼──────────────────┐
│     Cardano Blockchain           │
│----------------------------------│
│ - Aiken Validators               │
│ - Credential Registry            │
│ - Minting Policy (NFTs)          │
└──────────────────────────────────┘
                │
        ┌───────┴───────┬──────────────┐
        │               │              │
┌───────▼──────────┐ ┌──▼──────────┐ ┌──▼──────────┐
│ Backend API      │ │ Database    │ │ IPFS Storage │
│ (Node.js)        │ │ (Supabase)  │ │ (Pinata)     │
│------------------│ │-------------│ │-------------│
│ - Issuance       │ │ - Users     │ │ - Certificates│
│ - Validation     │ │ - Metadata  │ │ - PDFs       │
│ - QR Service     │ │ - Logs      │ │ - Hash refs  │
└──────────────────┘ └─────────────┘ └─────────────┘
```

3.2 Key Components

Component	Role
Frontend	Credential issuance + verification UI
Wallet	Signing issuance / verification transactions

Lucid	Transaction construction
Aiken Validators	Enforce credential integrity
Backend	Handles issuance workflows + QR generation
Supabase	Stores user + credential metadata
IPFS	Stores certificate files

4. Blockchain Design

4.1 Smart Contracts

Contract	Function
Credential Registry Validator	Stores credential state
Minting Policy	Issues credential NFTs
Access Control Logic	Restricts issuance to institutions

4.2 Credential Model

Each credential is represented as a **non-fungible token (NFT)** with:

- Unique credential ID
- Issuer address
- Metadata hash (IPFS reference)

4.3 Datum Structure

```
CredentialDatum {
 credential_id: ByteArray
 issuer: Address
 recipient: Address
 metadata_hash: ByteArray
 issued_at: Timestamp
 revoked: Bool
}
```

4.4 Redeemer Structure

```
IssueCredential
RevokeCredential
VerifyCredential
```

4.5 UTxO Model Usage

- Each credential = unique UTxO
- Issuance creates a new UTxO with credential metadata
- Revocation updates state (new UTxO with revoked = true)
- Ownership tied to recipient wallet

4.6 Token Management

Property	Value
Type	NFT

Minting	Only authorized issuers
Transfer	Optional (can be restricted)
Revocation	Controlled via validator

4.7 Validation Logic

```
validate_issue(tx):
 assert tx.signed_by(issuer)
 assert issuer in authorized_institutions
 assert metadata_hash exists

validate_verify(tx):
 assert hash(input_data) == datum.metadata_hash
```

4.8 Security Model

Risk	Mitigation
Fake credentials	Only authorized issuers
Data tampering	Hash verification (IPFS)
Replay attack	UTxO consumption

Unauthorized issuance	Signature + whitelist

5. Data Flow & Transaction Lifecycle

5.1 Issuance Flow

```
[1] University uploads credential data
[2] Data stored in IPFS → hash generated
[3] Backend constructs metadata
[4] Lucid builds issuance transaction
[5] Validator checks issuer authorization
[6] NFT minted → assigned to student wallet
```

5.2 Verification Flow

```
[1] Student shares QR code
[2] Verifier scans QR → fetches metadata
[3] Hash computed from credential
[4] Compare with on-chain datum
[5] If match → credential valid
```

5.3 Revocation Flow

```
[1] Issuer submits revoke transaction
[2] Validator verifies issuer identity
[3] Credential state updated (revoked = true)
```

6. Off-chain Components

6.1 Backend Services

Service	Function

Issuance Service	Creates credential metadata
Verification Service	Validates credentials
QR Generator	Encodes credential references
API Layer	Serves frontend

6.2 Data Storage

Data	Location
Credential files	IPFS
Metadata	Supabase
Proof reference	Blockchain

6.3 Integration

- Blockfrost → transaction submission
- IPFS (Pinata) → file storage
- Supabase → user + credential indexing

7. Sandbox / Testnet Results

7.1 Transaction Results

Tx ID	Type	Status	Notes
tx123	Issue	Success	NFT minted
tx456	Verify	Success	Hash matched
tx789	Revoke	Success	State updated

7.2 Performance Observations

- Validator execution within limits
- NFT minting stable
- No unauthorized issuance observed

8. Tools & Environment

Tool	Purpose
Cardano Preprod	Testnet
Aiken CLI	Smart contracts
Lucid	Transaction builder
Blockfrost	API layer

Supabase	Database
IPFS (Pinata)	Storage
Node.js	Backend
React	Frontend

9. Limitations

- No decentralized identity (DID) integration
- Manual issuer onboarding
- No zero-knowledge privacy layer
- Revocation requires issuer action (not automated)

10. Next Steps

10.1 Technical

- Introduce DID / Verifiable Credentials standard (W3C VC)
- Add ZK proofs for selective disclosure
- Optimize validator logic

10.2 Product

- Improve QR verification UX
- Add mobile-first interface
- Build credential wallet

10.3 Governance

- On-chain issuer registry
- Role-based access control

10.4 Deployment

- Move to mainnet-ready infra
- Conduct security audit

11. Summary

The prototype demonstrates a viable approach to decentralized credential verification using Cardano. By anchoring credential proofs on-chain and storing data off-chain, the system achieves a balance between privacy, scalability, and trust.

Further improvements are required to align with global standards (W3C VC, DID) and to enable production-grade deployment.